Entrega 3 – Estructuras de Datos

Ana Maria Murcia

Martin David Sanmiguel

Alejandro González Ochoa



Pontificia Universidad Javeriana

Facultad de ingeniería

24 de abril de 2026

Acta de evaluación

1 Entrega

Comentario de Andrea: Al momento de guardar, no se está preservando el orden original de los comandos, el cual es importante para garantizar un desplazamiento adecuado del robot

En la primera entrega del proyecto la clase Comando tenía una lista de CAnalisis y una lista de CMovimiento. Esto causaba problemas a la hora de leer un archivo de comandos pues para cada comando se tenía que verificar qué tipo de comando era antes de guardarlo. Adicionalmente, la clase Curiosity hacía todo: procesaba los argumentos del usuario, daba las funciones de ayuda, utilizaba distintas funciones utilitarias en distintas funciones de comandos y ejecutaba todas las funciones de comandos del sistema.

Para solucionar este problema se implementó una herencia de la clase Comando hacia las clases CAnalisis y CMovimiento. No solo se hizo el cambio de implementar herencia, también se separaron responsabilidades. El cambio se refleja en que ahora Curiosity solo maneja las funciones de comandos del sistema, pues se hizo una clase aparte llamada Utilities, la cual solo tiene funciones estáticas que se utilizan a lo largo del sistema. Adicionalmente, se separó el procesamiento de los argumentos del usuario en la clase InterfazUsuario, la cual realiza verificaciones básicas y llama las funciones de Curiosity.

El orden al guardar los comandos se logra haciendo que la clase Curiosity tuviera una cola de Comando en la cual se guardan todos los comandos tanto de movimiento como de análisis. Esto es posible porque la cola es polimórfica, es decir, al ser de Comando, la lista puede guardar cualquiera de sus clases hijas (CMovimiento y CAnalisis), lo que garantiza que el orden de inserción de los comandos sea el mismo de ejecución.

Es posible guardar cualquiera de estas tres clases debido a que la cola guarda apuntadores a Comando, lo que le permite guardar objetos de distinto tamaño, lo cual es crucial pues tanto CMovimiento como CAnalisis son objetos más grandes que Comando. Se escogió una cola pues nos pareció la mejor forma de simularle una memoria al Curiosity.

2 Entrega

Comentario de Andrea: El diseño debería describir toda la información del TAD, si sólo se incluyen operaciones no es un TAD, si sólo se incluyen datos tampoco es un TAD.

En la segunda entrega se tenían 2 tads que no corresponden a la definición de este tipo de dato, Utilities y interfazUsuario. Utilities almacenaba todas las funciones estáticas de ayuda, que se usan a lo largo de todo el sistema pero no poseía datos mínimos e interfazUsuario que solo almacena datos de ejecución pero no tenía operaciones.

Por lo tanto estas clases no son consideradas Tads y fueron eliminadas tanto del documento de diseño como del diagrama de relación.

Comentario de Andrea: en_cuadrante debería forzar que cuando se carga un nuevo archivo se ubiquen nuevamente los elementos, no mostrar la información del archivo (árbol) anterior.

En la segunda entrega en el comando de en_cuadrante cada que se cargaba un archivo el sistema seguía teniendo la información del archivo anterior almacenada, esta se mezclaba con el archivo actual y generaba datos incorrectos.

Para solucionar este problema en la tercera entrega el comando crea el árbol con cada archivo nuevo en lugar de agregar los elementos a un árbol ya existente.

Comentario de Andrea: guardar_comandos y guardar_elementos imprime información que no corresponde con la ejecución del comando. Y No deberían imprimir el recorrido del árbol en ubicar_elementos, no es necesario.

Anteriormente se imprimían tanto el recorrido completo del árbol para verificar su funcionamiento como mensajes de ayuda innecesarios en el comando guardar_comandos y ubicar_elementos.

Para esta tercera entrega eliminamos todos estos mensajes para una ejecución más limpia.

TADS

TAD NodoKD

Datos minimos

- elemento, Elemento, representa la información almacenada en el nodo, como un punto o componente dentro del espacio.

- hijoIzq, puntero a NodoKD, referencia al subárbol izquierdo del nodo, utilizado para organizar los elementos según una dimensión.

- hijoDer, puntero a NodoKD, referencia al subárbol derecho del nodo, utilizado para organizar los elementos según otra partición del espacio.

Operaciones

- obtenerElemento(), retorna el elemento almacenado en el nodo.

- fijarElemento(val), permite asignar o modificar el elemento del nodo.

- obtenerHijoIzq(), retorna el hijo izquierdo del nodo.

- obtenerHijoDer(), retorna el hijo derecho del nodo.

- fijarHijoIzq(hijo), permite asignar o modificar el hijo izquierdo del nodo.

- fijarHijoDer(hijo), permite asignar o modificar el hijo derecho del nodo.

- esHoja(), retorna un valor booleano que indica si el nodo no tiene hijos, es decir, si es un nodo hoja.

TAD KDTree

Datos minimos

- Raiz, puntero a NodoKD, representa el nodo principal del árbol k-d desde el cual se organizan todos los elementos en el espacio.

Operaciones

- esVacio(), retorna un valor booleano que indica si el árbol no contiene elementos.

- obDatoRaiz(), retorna el elemento almacenado en la raíz del árbol.

- altura(), retorna la altura total del árbol.

- altura(nodo), calcula y retorna la altura del subárbol cuya raíz es el nodo dado.

- tamano(), retorna la cantidad total de nodos en el árbol.

- tamano(nodo), calcula y retorna la cantidad de nodos en el subárbol cuya raíz es el nodo dado.

- insertarNodo(val), inserta un nuevo elemento en el árbol según las reglas del árbol k-d y retorna si la operación fue exitosa.

- eliminarNodo(val), elimina un elemento del árbol y retorna si la operación fue exitosa.

- buscarNodo(val), busca un nodo que contenga el elemento dado y retorna una referencia a dicho nodo si existe.

- preOrden(), retorna una colección de elementos recorriendo el árbol en preorden.

- preOrden(nodo, valores), realiza el recorrido en preorden desde un nodo dado y almacena los elementos en la colección proporcionada.

- inOrden(), retorna una colección de elementos recorriendo el árbol en inorden.

- inOrden(nodo, valores), realiza el recorrido en inorden desde un nodo dado y almacena los elementos en la colección proporcionada.

- posOrden(), retorna una colección de elementos recorriendo el árbol en posorden.

- posOrden(nodo, valores), realiza el recorrido en posorden desde un nodo dado y almacena los elementos en la colección proporcionada.

- nivelOrden(), retorna una colección de elementos recorriendo el árbol por niveles.

- encontrarPadreMayor(nodo), retorna el nodo padre del mayor elemento dentro de un subárbol.

- enCuadrante(x1, y1, x2, y2), retorna una colección de elementos que se encuentran dentro de un rango de coordenadas definido por dos puntos.

- enCuadrante(val, nodo, elementos, x1, y1, x2, y2), recorre el árbol desde un nodo dado y almacena en una colección los elementos que se encuentran dentro del cuadrante especificado.

- clear(), elimina todos los nodos del árbol, dejándolo vacío.

TAD Elemento

Datos minimos

- tipo_comp, cadena de caracteres, tipo de componente: roca, cráter, montículo o duna

- tamaño, número real, da cuenta de qué tan grande es el elemento.

- unidad, cadena de caracteres, cm, dm, m, km .

- cord_x, número real, representa la coordenada x en el plano cartesiano.

- cord_y, número real, representa la coordenada y en el plano cartesiano.

Operaciones

- obtenerTipo_comp(), retorna el tipo de componente del elemento.

- obtenerTamano(), retorna el tamaño del elemento.

- obtenerUnidad(), retorna la unidad de medida del elemento.

- obtenerCord_x(), retorna la coordenada x en el plano cartesiano.

- obtenerCord_y(), retorna la coordenada y en el plano cartesiano.

- fijarTipo(n_tipo), cambia el tipo de componente del elemento.

- fijarTamano(n_tamano), cambia el tamaño del objeto.

- fijarUnidad(n_unidad), cambia la unidad de medida del elemento.

- fijarCord_x(n_cord_x), cambia retorna la coordenada x en el plano cartesiano.

- fijarCord_y(n_cord_y), cambia retorna la coordenada y en el plano cartesiano.

TAD Grafo

Datos minimos

- vertices, vector de vertices, representa el conjunto de vértices almacenados en el grafo.

- aristas, lista de aristas, representa las conexiones entre vértices y el costo asociado a cada conexión mediante una lista de adyacencia.

Operaciones

- obtenerVertices(), retorna el conjunto actual de vértices del grafo.

- obtenerAristas(), retorna la estructura de adyacencia que almacena las conexiones del grafo.

- cantidadVertices(), retorna la cantidad de vértices almacenados en el grafo.

- cantidadAristas(), retorna la cantidad total de aristas existentes en el grafo.

- buscarVertice(vertice), retorna la posición del vértice indicado dentro de la estructura del grafo.

- costo(vOri, vDes), retorna el costo de la arista que conecta los vértices origen y destino.

- insertarVertice(vertice), agrega un nuevo vértice al grafo.

- insertarArista(vOri, vDes, cost), agrega una nueva arista entre los vértices origen y destino con el costo especificado.

- eliminarVertice(vertice), elimina el vértice indicado y todas sus conexiones asociadas.

- eliminarArista(vOri, vDes), elimina la arista existente entre los vértices origen y destino.

- recorridoPlano(), retorna una colección con todos los vértices almacenados en el grafo.

- recorridoDFS(vertice), realiza un recorrido en profundidad a partir del vértice indicado y retorna los vértices visitados.

- recorridoBFS(vertice), realiza un recorrido en anchura a partir del vértice indicado y retorna los vértices visitados.

- dijkstra(indiceVerticeInicio), calcula los caminos mínimos desde el vértice de inicio hacia los demás vértices del grafo.

- vacio(), verifica si el grafo contiene o no vértices almacenados.

- construirCamino(origen, destino, predecesores), construye y retorna el camino entre un vértice origen y un vértice destino utilizando la información de predecesores suministrada.

TAD CAnalisis

Datos minimos:

- objeto, cadena de caracteres, el nombre del elemento que se quiere analizar (roca, arena, monticulo, ...).

- comentario, cadena de caracteres, comentario del objeto.

Operaciones:

- obtenerTipoAnalisis, retorna el tipo de análisis almacenado.

- obtenerObjeto(), retorna el nombre del tipo de objeto analizado.

- obtenerComentario(), retorna el comentario del analisis.

- fijarTipoAnalisis, asigna el tipo de análisis.

- fijarObjeto(nObj), cambia el nombre del tipo de objeto analizado.

- obtenerComentario(nCom), cambia el comentario del objeto.

TAD CMovimiento

Datos minimos

- Mag, numero real, si es avanzar, el número de unidades que se espera avanzar, si es girar la cantidad de grados que debe girar.

- Unidad, cadena de caracteres, cm, dm, m, km para avanzar y grd, rad para girar

Operaciones
- obtenerTipoMovimiento(), retorna el tipo de movimiento.

- obtenerMagnitud(), retorna la magnitud del movimiento.

- obtenerUnidad(), retorna la unidad del movimiento

- fijarTipoMovimiento(n_tipo), cambia el tipo de movimiento.

- fijarMagnitud(n_magnitud), cambia la magnitud del movimiento.

- fijarUnidad(n_unidad), cambia la unidad del movimiento.

- fijarTipoMovimiento(nTipoMovimiento), asigna el tipo de movimiento.

TAD Comando

Datos minimos:
- tipoAccion, cadena de caracteres, , representa el tipo de acción que describe el comando,, movimiento(girar o avanzar) o análisis.

Operaciones:
- obtenerTipoAccion(), retorna el tipo de acción almacenado en el comando.

- fijarTipoAccion(n_tipoAccion), asigna el tipo de acción del comando.

TAD Curiosity

Datos minimos:
- elementos, elemento, representa el conjunto de elementos almacenados en el sistema, como rocas, dunas u otros objetos.

- comandos,comando , almacena los comandos que se ejecutarán en orden de llegada.

- arbolElementos, KDTree, organiza los elementos en el espacio para facilitar búsquedas por coordenadas.

- Grafo, Grafo, representa el grafo formado a travez de las conecciones de los elementos guardados.

Operaciones:

- obtenerElementos(), retorna la lista de elementos.
- obtenerComandos(), retorna el objeto de comando.
- obtenerArbolElementos(), retorna el árbol k-d que organiza los elementos.
- obtenerGrafo(), retorna el grafo que organiza los elementos
- fijarComandos(n_comandos), cambia la lista de comandos.
- fijarElementos(n_elementos), cambia la lista de elementos.
- fijarArbolElementos(n_arbol), cambia el árbol de elementos.
- Ssd
- fijarGrafo(n_grafo), cambia el grafode elementos.
- agregarMovimiento(args), crea y agrega un comando de movimiento a partir de los argumentos proporcionados.
- agregarAnalisis(args, comentario), crea y agrega un comando de análisis con los argumentos y un comentario asociado.
- limpiarComandos(), elimina todos los comandos almacenados en la cola.
- cargar_comandos(args), carga un conjunto de comandos desde una fuente externa utilizando los argumentos dados.
- cargar_elementos(args), carga un conjunto de elementos desde una fuente externa.
- agregar_movimiento(args, imprimir), agrega un comando de movimiento y opcionalmente imprime información del proceso.
- agregar_analisis(args, imprimir), agrega un comando de análisis y opcionalmente imprime información.
- agregar_elemento(args, imprimir), crea y agrega un nuevo elemento al sistema y opcionalmente muestra información.
- guardar(args), guarda el estado actual de los elementos o comandos en una fuente externa.
- simular_comandos(args), simula la ejecución de los comandos almacenados.

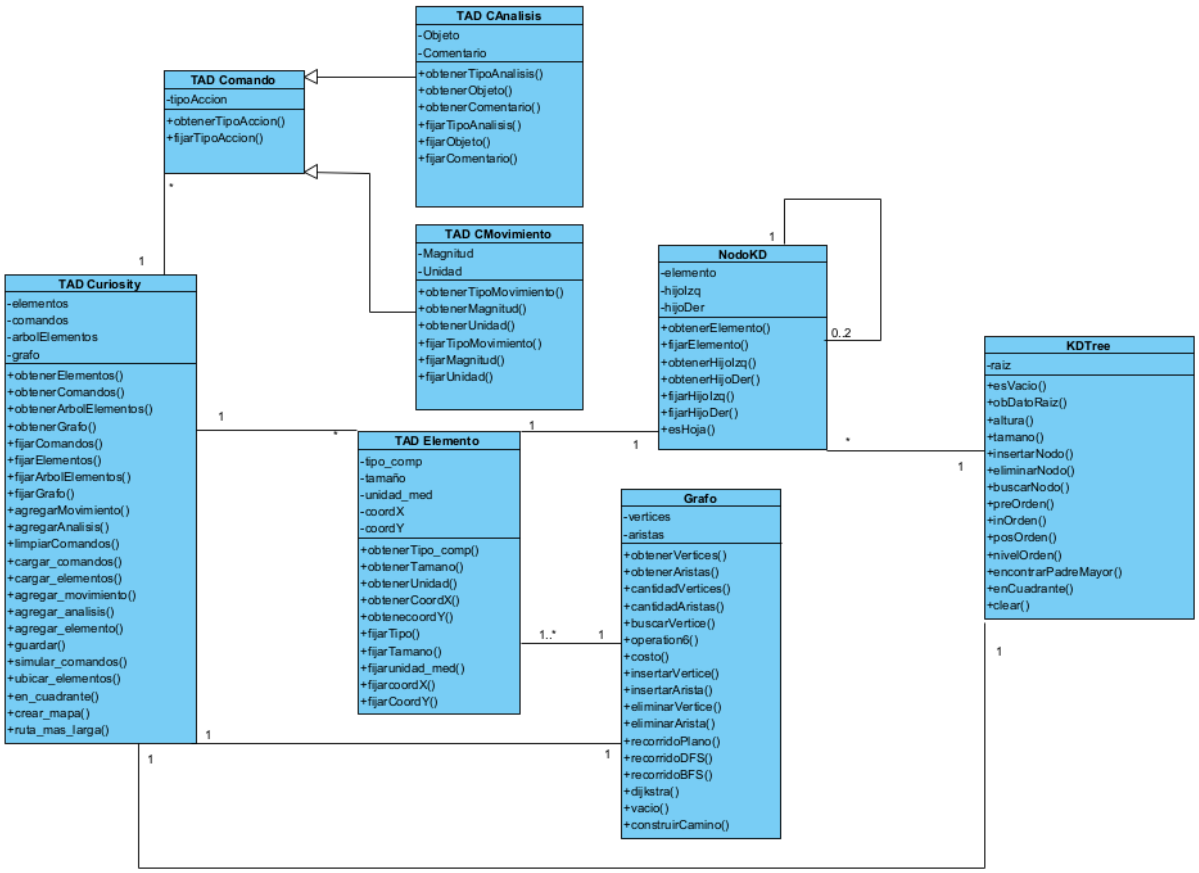
- ubicar_elementos(), posiciona los elementos dentro del árbol según sus coordenadas.

- en_cuadrante(args), consulta los elementos que se encuentran en un cuadrante específico.

- crear_mapa(args), genera una representación del entorno a partir de los elementos almacenados.

- ruta_mas_larga(), calcula la ruta más larga posible basada en los comandos o elementos disponibles.

Diagrama de relación



Diseño

cargar_comandos

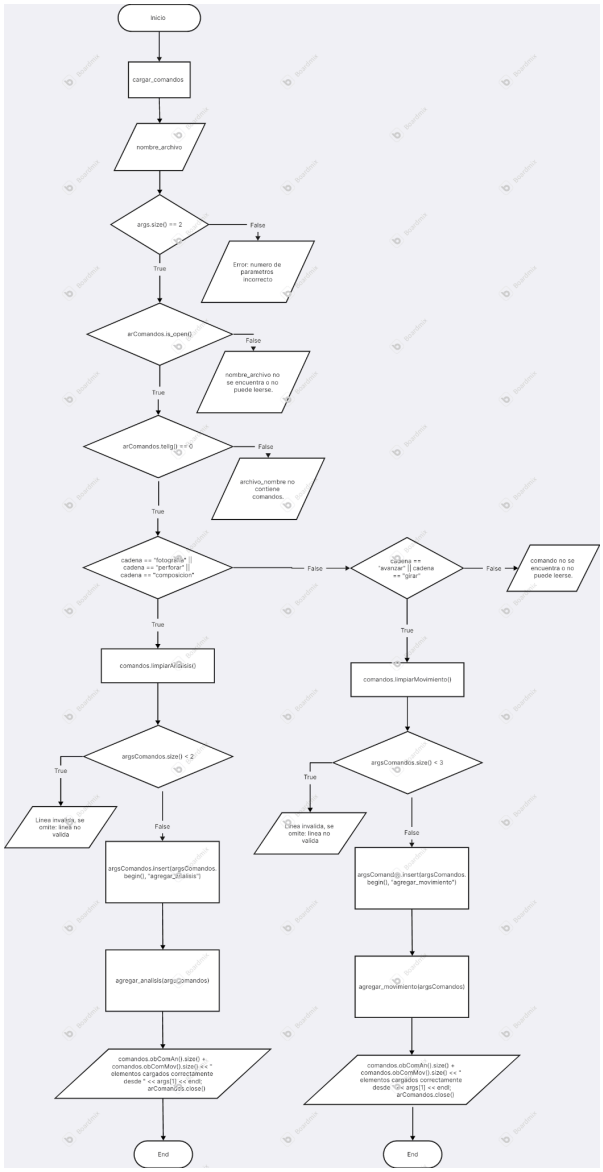
-Entradas: nombre_archivo.

-Salidas: Carga los comandos al contenedor de comandos almacenados. Aparte de esta acción muestra 6 diferentes tipos de mensajes en pantalla dependiendo de las entradas

- Archivo no contiene comandos.
- Linea invalida, se omite: linea invalida.
- Commando agregar_analisis
- Comando agregar_movimiento.
- Nombre_archivo no se encuentra o no puede leerse.
- Error: numero de parametros incorrecto

-Condiciones para el comando:
No debe llenar más información aparte de los parámetros solicitados.

Diagrama de flujo UML



cargar_elementos

-Entradas: cargar_elementos, nombre de un archivo.

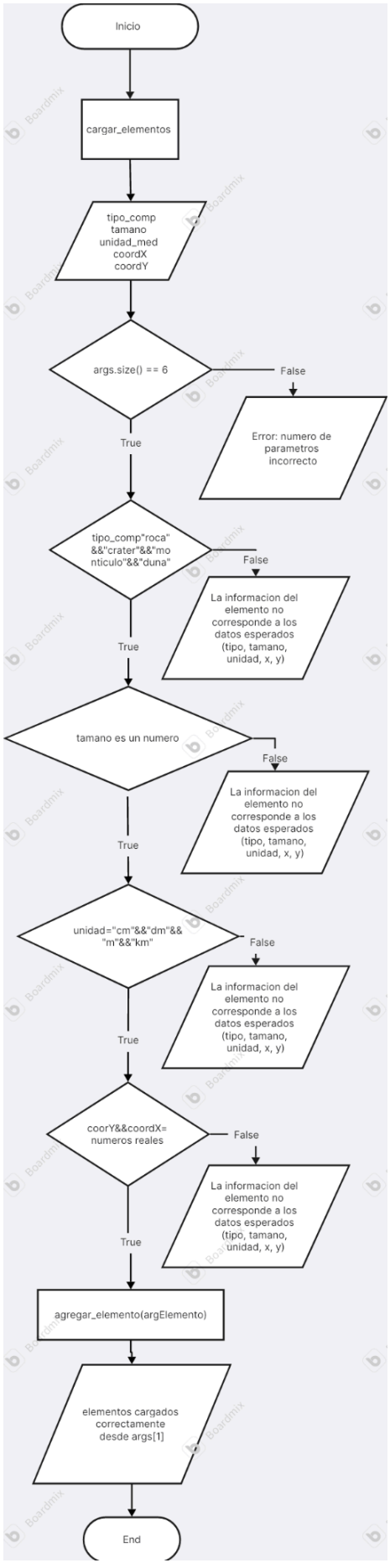
-Salidas: Carga los elementos al contenedor de elementos almacenados. Aparte de esta acción muestra 6 diferentes tipos de mensajes en pantalla dependiendo de las entradas:

- nombre_archivo no contiene elementos.
- Linea invalida, se omite: linea con formato incorrecto.
- Elemento dentro del archivo fue agregado.
- Número de elementos cargados correctamente desde nombre_archivo.
- nombre_archivo no se encuentra o no puede leerse.

- Error: numero de parametros incorrecto

-Condiciones para el comando:
No debe llenar más información aparte de los parámetros solicitados.
En el archivo la información debe seguir la estructura tipo_componente tamaño unidad_med coordX coordY.

Diagrama de flujo UML


Inicio
cargar_elementos
tipo_comp
tamano
unidad_med
coordX
coordY
args.size() == 6
False
Error: numero de parametros incorrecto
True
tipo_comp"roca"&&"crater"&&"monticulo"&&"duna"
False
La informacion del elemento no corresponde a los datos esperados (tipo, tamano, unidad, x, y)
True
tamano es un numero
False
La informacion del elemento no corresponde a los datos esperados (tipo, tamano, unidad, x, y)
True
unidad="cm"&&"dm"&&"m"&&"km"
False
La informacion del elemento no corresponde a los datos esperados (tipo, tamano, unidad, x, y)
True
coorY&&coordX= numeros reales
False
La informacion del elemento no corresponde a los datos esperados (tipo, tamano, unidad, x, y)
True
agregar_elemento(argElemento)
elementos cargados correctamente desde args[1]
End

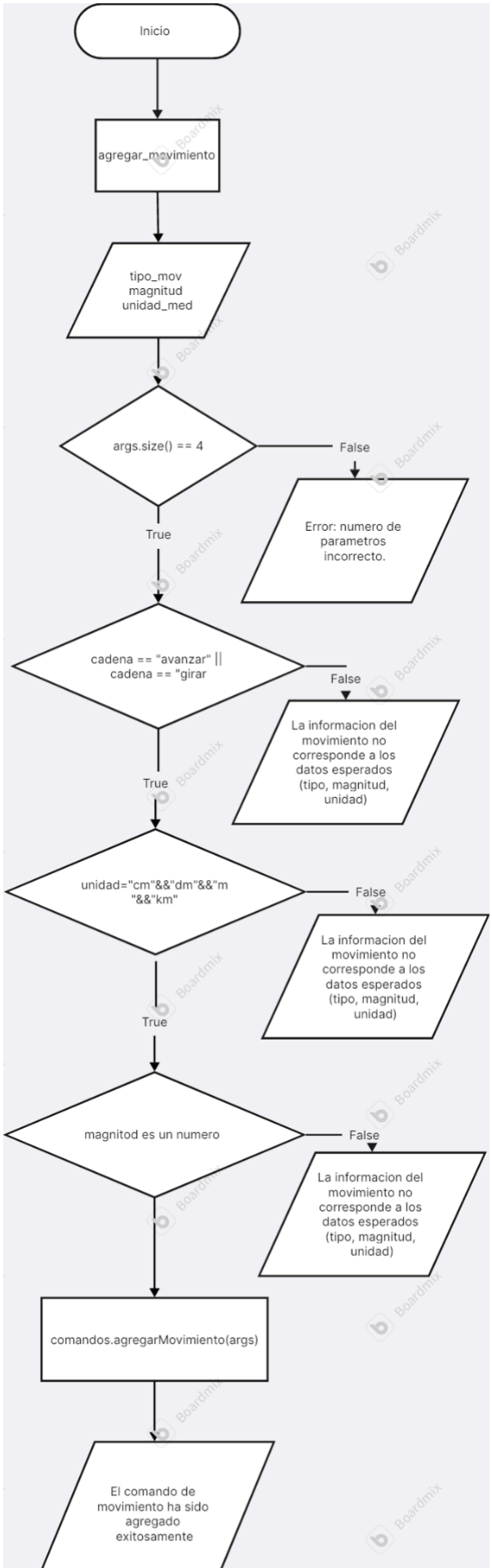
agregar_movimiento

-Entradas: tipo_mov magnitud unidad_med
-Salidas: Agrega los comandos de movimiento al contenedor de comandos. Aparte de esta acción muestra 3 diferentes tipos de mensajes en pantalla dependiendo de las entradas:

- La informacion del movimiento no corresponde a los datos esperados (tipo, magnitud, unidad).
- El comando de movimiento ha sido agregado exitosamente
- Error: numero de parametros incorrecto

-Condiciones para el comando:
No debe llenar más información aparte de los parámetros solicitados.
El comando debe seguir la estructura tipo_mov magnitud unidad_med

Diagrama de flujo UML


Inicio
agregar_movimiento
tipo_mov
magnitud
unidad_med
args.size() == 4
False
Error: numero de parametros incorrecto.
True
cadena == "avanzar" || cadena == "girar
False
La informacion del movimiento no corresponde a los datos esperados (tipo, magnitud, unidad)
True
unidad="cm"&&"dm"&&"m "&&"km"
False
La informacion del movimiento no corresponde a los datos esperados (tipo, magnitud, unidad)
True
magnitod es un numero
False
La informacion del movimiento no corresponde a los datos esperados (tipo, magnitud, unidad)
comandos.agregarMovimiento(args)
El comando de movimiento ha sido agregado exitosamente

agregar_analisis

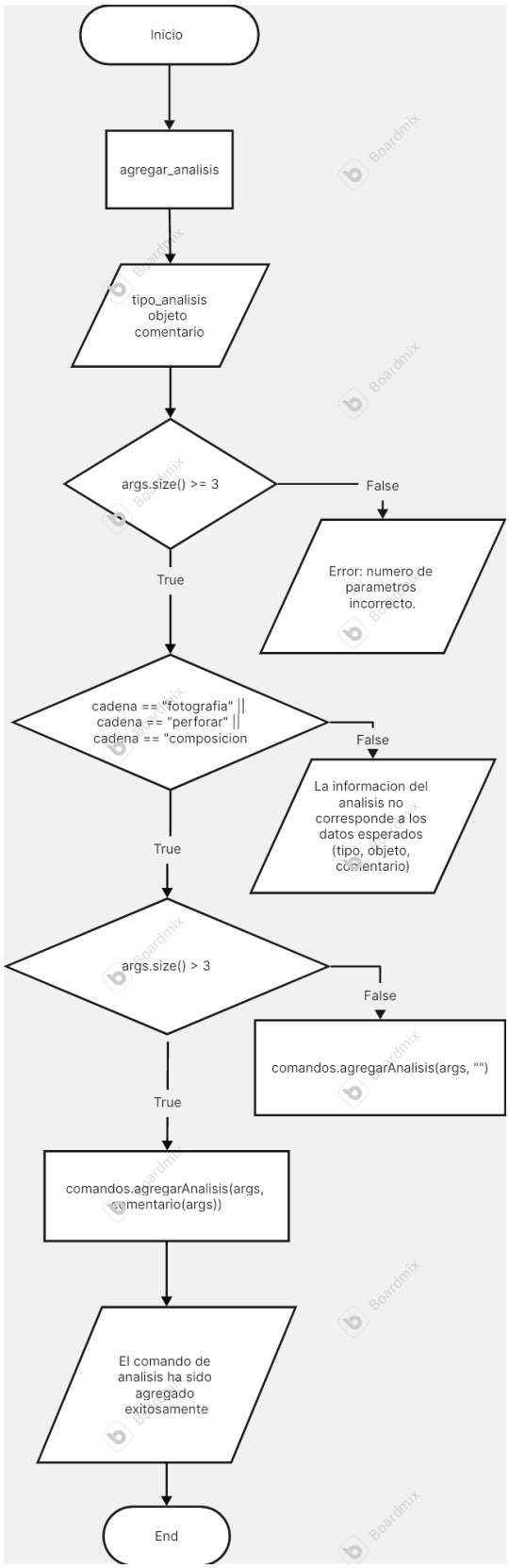
-Entradas: tipo de analisis, objeto y opcionalmente comentario.

-Salidas: agrega un analisis al contenedor de comandos de analisis
Aparte de esta acción muestra 3 diferentes tipos de mensajes en pantalla dependiendo de las entradas:

- La información del analisis no corresponde a los datos esperados (tipo, objeto, comentario)
- El comando de analisis ha sido agregado exitosamente.
- Error: numero de parametros incorrecto

-Condiciones para el comando:
No debe llenar más información aparte de los parámetros solicitados.
El comentario debe estar encerrado en comillas simples.
El comando de analisis a ingresar debe seguir la estructura tipo_analisis objeto comentario

Diagrama de flujo UML


Inicio
agregar_analisis
tipo_analisis
objeto
comentario
args.size() >= 3
False
Error: numero de parametros incorrecto.
True
cadena == "fotografia" || cadena == "perforar" || cadena == "composicion
False
La informacion del analisis no corresponde a los datos esperados (tipo, objeto, comentario)
True
args.size() > 3
False
comandos.agregarAnalisis(args, "")
True
comandos.agregarAnalisis(args, comentario(args))
El comando de analisis ha sido agregado exitosamente
End

agregar_elemento

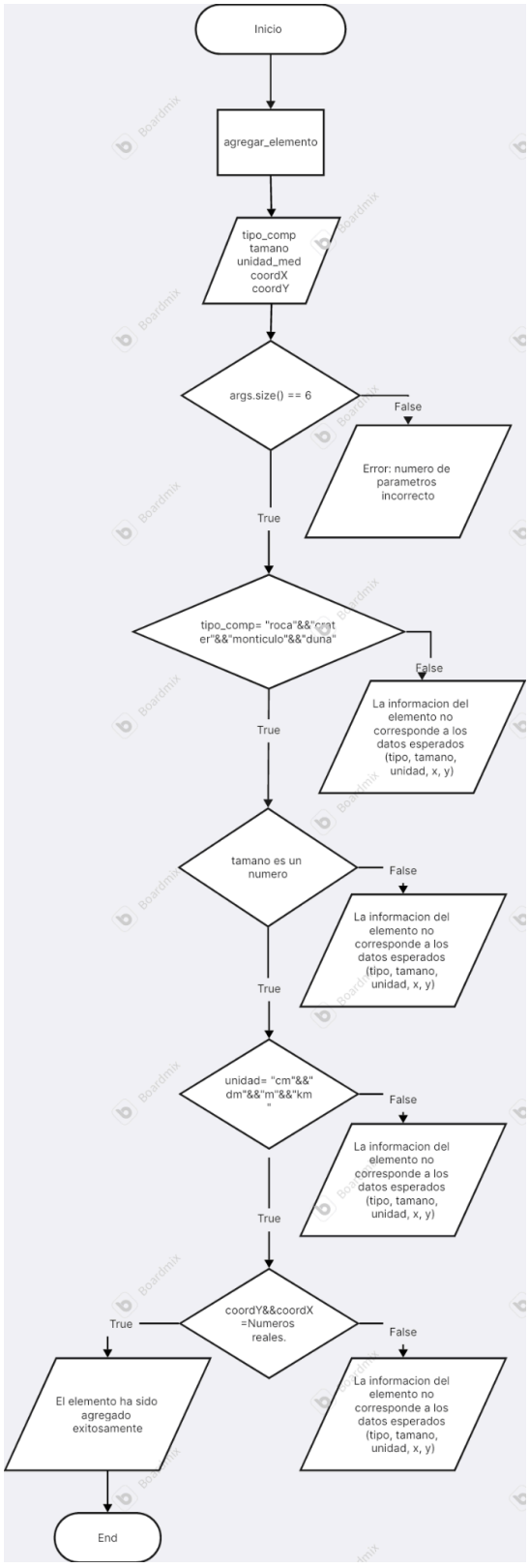
-Entradas: tipo de comp tamaño unidad_med coordX coordY.

-Salidas: agrega un elemento al contenedor de elementos.
Aparte de esta acción muestra 3 diferentes tipos de mensajes en pantalla dependiendo de las entradas:

- La información del elemento no corresponde a los datos esperados (tipo, tamano, unidad, x, y)
- El elemento ha sido agregado exitosamente
- Error: numero de parametros incorrecto

-Condiciones para el comando:
No debe llenar más información aparte de los parámetros solicitados.
El elemento a agregar debe seguir la estructura tipo_componente tamaño unidad_med coordX coordY.

Diagrama de flujo UML



guardar

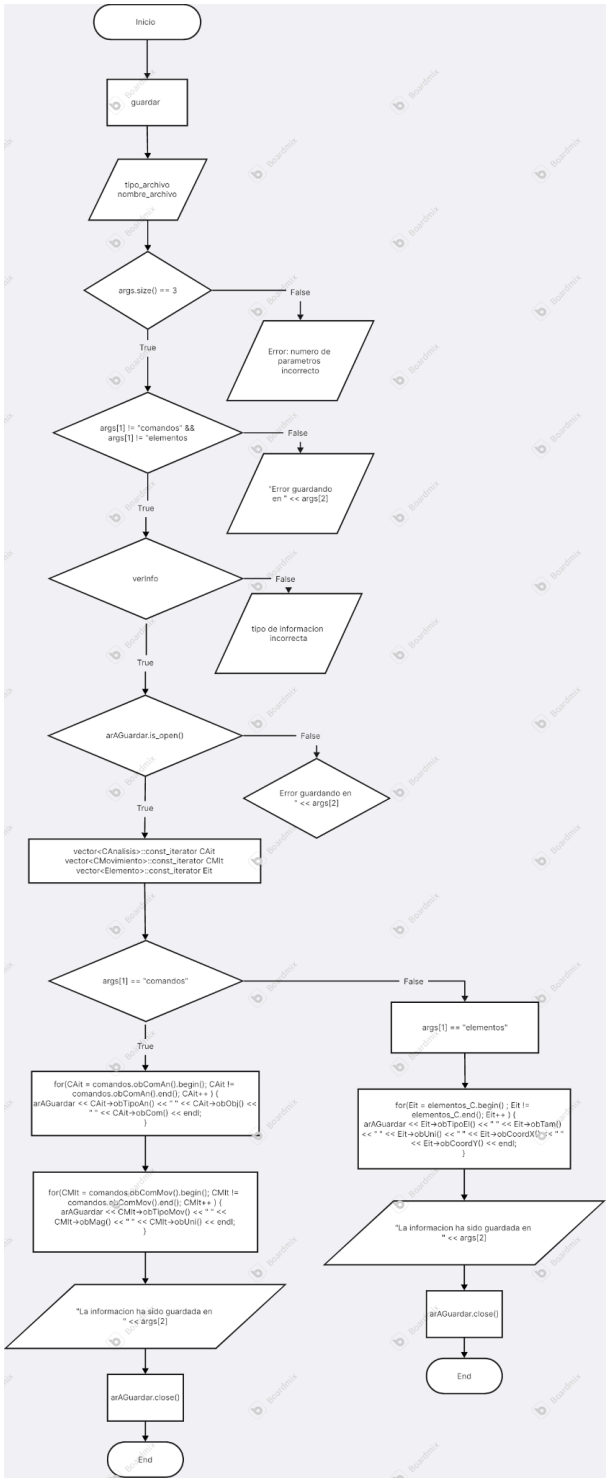
-Entradas: tipo_archivo nombre_archivo

-Salidas: Carga los comandos al contenedor de comandos almacenados.
Aparte de esta acción muestra 6 diferentes tipos de mensajes en pantalla dependiendo de las entradas

- Archivo no contiene comandos.
- Linea invalida, se omite: linea invalida.
- Commando agregar_analisis
- Comando agregar_movimiento.
- Nombre_archivo no se encuentra o no puede leerse.
- Error: numero de parametros incorrecto

-Condiciones para el comando:
No debe llenar más información aparte de los parámetros solicitados.
El elemento a agregar debe seguir la estructura tipo_archivo nombre_archivo.

Diagrama de flujo UML



simular_comandos

-Entradas: coordX coordY

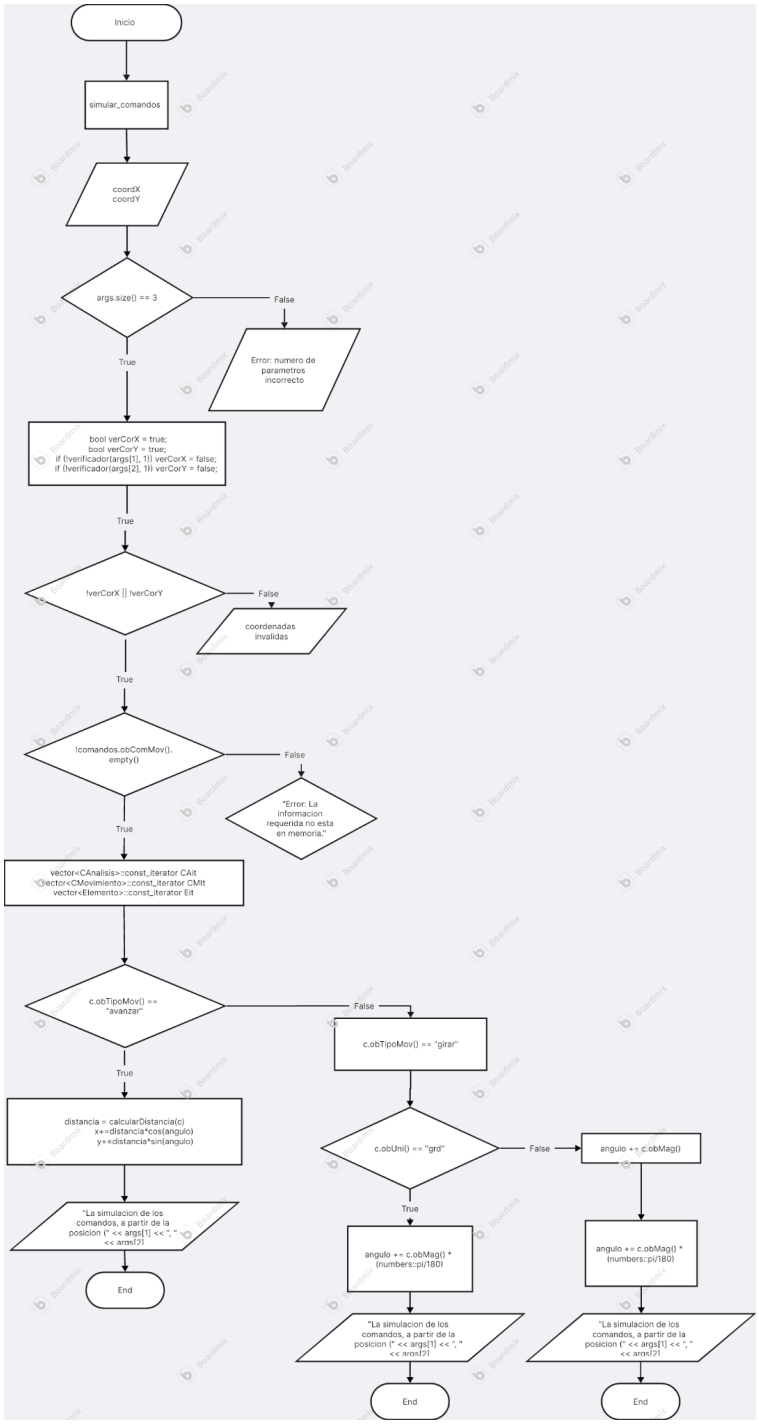
-Salidas: Retorna la posición x y la posición y final desl curiosity despues de ejecutar los
comandos

Aparte de esta acción muestra 8 diferentes tipos de mensajes en pantalla dependiendo de las entradas

- La simulación de los comandos, a partir de la posición x,y), deja al robot en la nueva posición ("x,y)."
- Error: La información requerida no esta en memoria.
- coordenadas invalidas
- Error: numero de parametros incorrecto
- tipo de información incorrecta
- Error guardando en nombre_archivo
- La información ha sido guardada en nombre_archivo

-Condiciones para el comando:
Se deben tener comandos en memoria.
No debe llenar más información aparte de los parámetros solicitados.
El elemento a agregar debe seguir la estructura coordX coordY.

Diagrama de flujo UML



Salir

-Entradas: Salir
-Salidas: Se termina de ejecutar el programa.
-Condiciones para el comando: N/A

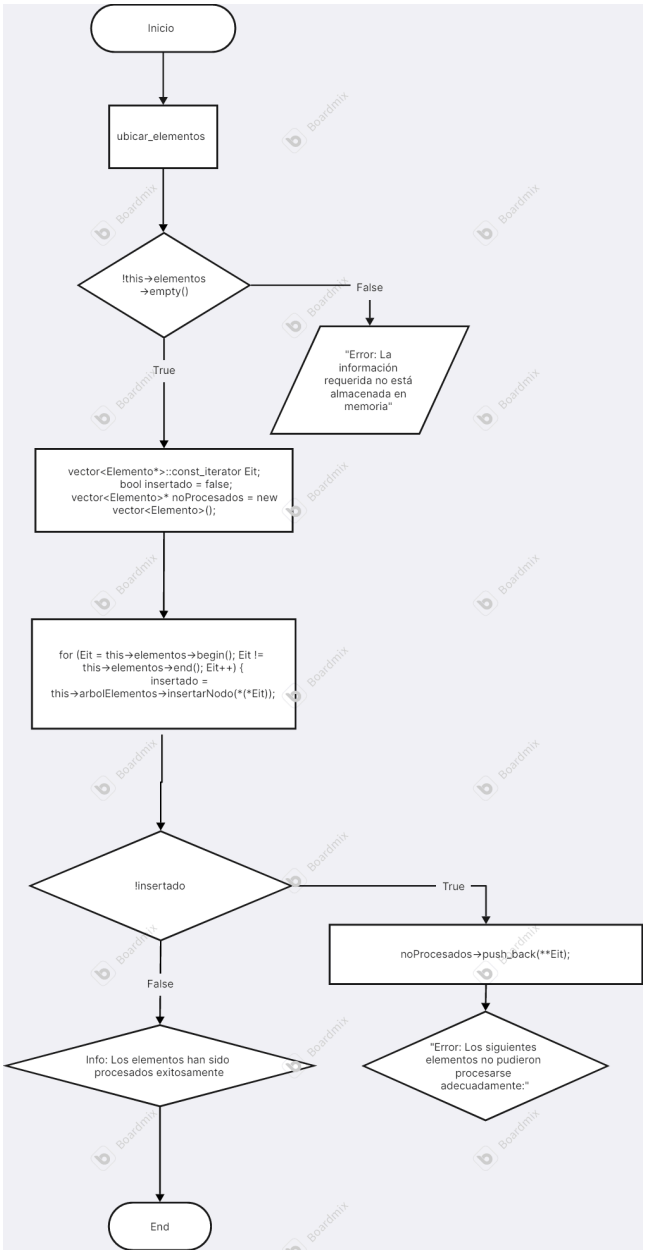
ubicar_elementos

-Entradas: solo el comando
-Salidas: todos los elementos guardados en memoria
Aparte de esta acción muestra 3 diferentes tipos de mensajes en pantalla dependiendo de las entradas

- "Error: La información requerida no está almacenada en memoria"
- Error: Los siguientes elementos no pudieron procesarse adecuadamente
- Info: Los elementos han sido procesados exitosamente"

-Condiciones para el comando:
Se debe haber agregado elementos con anterioridad.
No debe llenar más información aparte de los parámetros solicitados.

Diagrama de flujo UML



en_cuadrante

-Entradas: coordX1 coordX2 coordY1 coordY2

-Salidas: los elementos en el cuadrante solicitado

Aparte de esta acción muestra 7 diferentes tipos de mensajes en pantalla dependiendo de las entradas

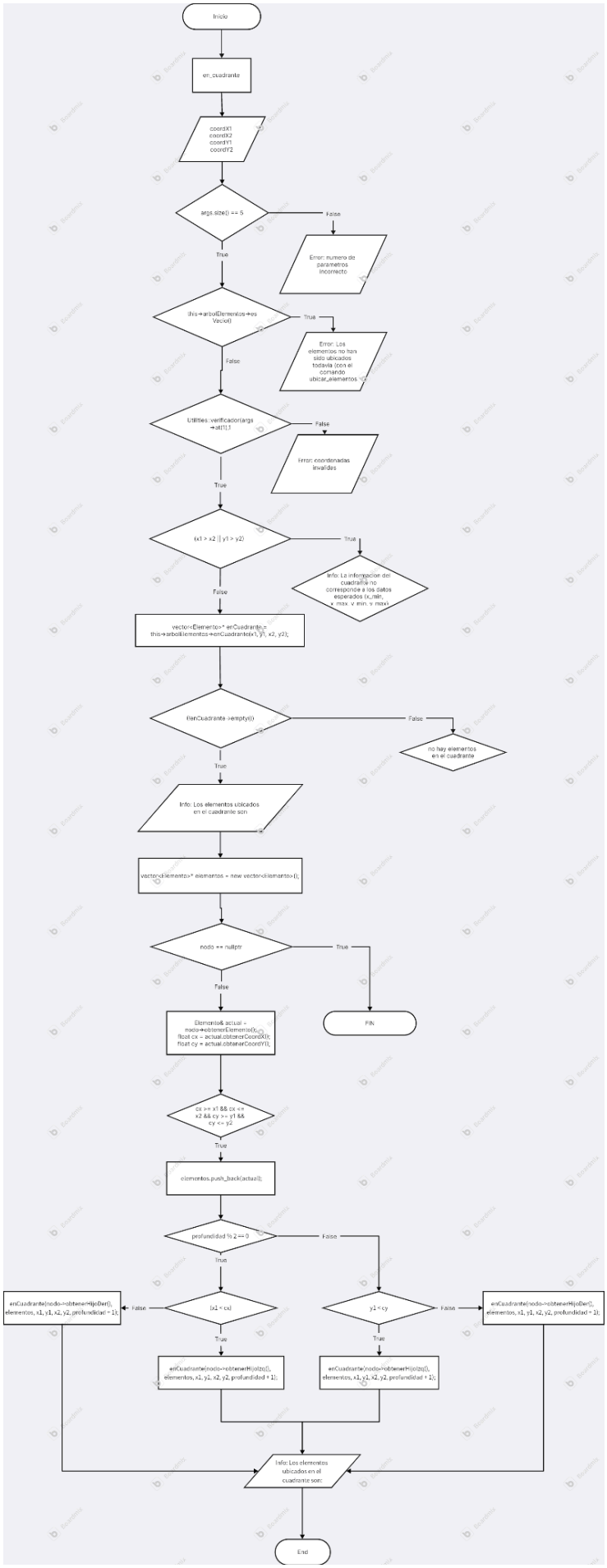
- Error: numero de parametros incorrecto
- Error: Los elementos no han sido ubicados todavia (con el comando ubicar_elementos
- Error: coordenadas invalidas
- Info: La informacion del cuadrante no corresponde a los datos esperados (x_min, x_max, y_min, y_max)

- no hay elementos en el cuadrante
- Info: Los elementos ubicados en el cuadrante son:

-Condiciones para el comando:

Se debe haber agregado elementos con anterioridad.
No debe llenar más información aparte de los parámetros solicitados.
Debe seguir la estructura en_cuadrante coordX1 coordX2 coordY1 coordY2

Diagrama de flujo UML


Inicio
en_cuadrante
coordX1
coordX2
coordY1
coordY2
args.size() == 5
False
Error: numero de parametros incorrecto
True
this->arbolElementos->esVacio()
True
Error: Los elementos no han sido ubicados todavia (con el comando ubicar_elemantos
False
Utilities::verificador(args->at(1),1
False
Error: coordenadas invalidas
True
(x1 > x2 || y1 > y2)
True
Info: La informacion del cuadrante no corresponde a los datos esperados (x_min, x_max, y_min, y_max)
False
vector<Elemento>* enCuadrante = this->arbolElementos->enCuadrante(x1, y1, x2, y2);
!enCuadrante->empty()
False
no hay elementos en el cuadrante
True
Info: Los elementos ubicados en el cuadrante son
vector<Elemento>* elementos = new vector<Elemento>();
nodo == nullptr
True
FIN
False
Elemento& actual = nodo->obtenerElemento(); float cx = actual.obtenerCoordX(); float cy = actual.obtenerCoordY();
cx >= x1 && cx <= x2 && cy >= y1 && cy <= y2
True
elementos.push_back(actual);
profundidad % 2 == 0
False
True
(x1 < cx)
False
enCuadrante(nodo->obtenerHijoDer(), elementos, x1, y1, x2, y2, profundidad + 1);
True
enCuadrante(nodo->obtenerHijoIzq(), elementos, x1, y1, x2, y2, profundidad + 1);
y1 < cy
False
enCuadrante(nodo->obtenerHijoDer(), elementos, x1, y1, x2, y2, profundidad + 1);
True
enCuadrante(nodo->obtenerHijoIzq(), elementos, x1, y1, x2, y2, profundidad + 1);
Info: Los elementos ubicados en el cuadrante son:
End

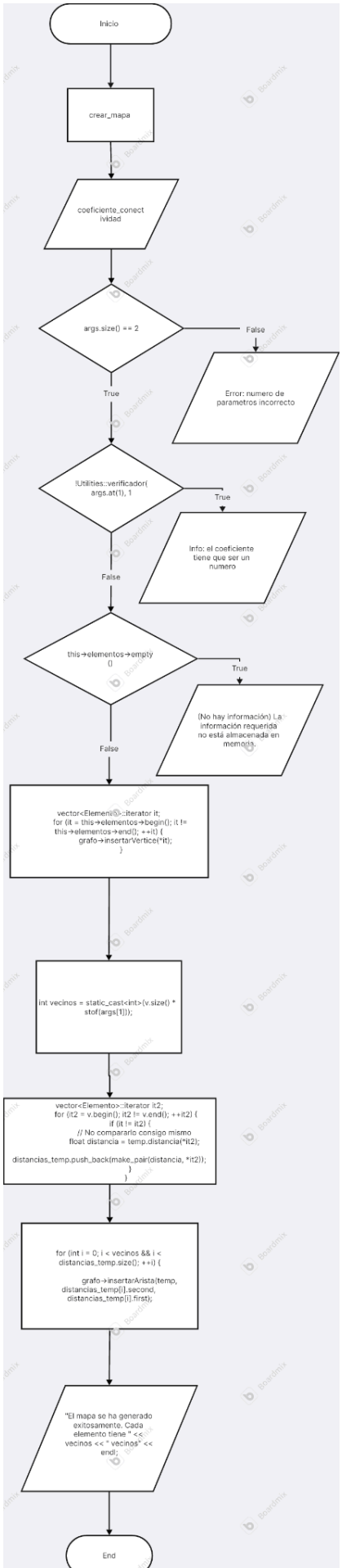
crear_mapa

-Entradas:coeficiente_conectividad
-Salidas: nada
Aparte de esta acción muestra 7 diferentes tipos de mensajes en pantalla dependiendo de las entradas

- Info: el coeficiente tiene que ser un numero
- (No hay información) La información requerida no está almacenada en memoria.
- El mapa se ha generado exitosamente. Cada elemento tiene
- "Error: numero de parametros incorrecto
- (No hay información) El mapa no ha sido generado todavía (con el comando crear_mapa)
- (No hay información) No hay rutas válidas en el mapa.
- (Resultado exitoso) Los puntos de interés más alejados entre sí son

-Condiciones para el comando:
Se debe haber agregado elementos con anterioridad.
No debe llenar más información aparte de los parámetros solicitados.
Debe seguir la estructura crear_mapa coeficiente_conectividad

Diagrama de flujo UML


Inicio
crear_mapa
coeficiente_conectividad
args.size() == 2
False
Error: numero de parametros incorrecto
True
!Utilities::verificador(args.at(1), 1
True
Info: el coeficiente tiene que ser un numero
False
this->elementos->empty()
True
(No hay información) La información requerida no está almacenada en memoria
False
vector<Elemento>::iterator it;
for (it = this->elementos->begin(); it != this->elementos->end(); ++it) {
grafo->insertarVertice(*it);
}
int vecinos = static_cast<int>(v.size() * stof(args[1]));
vector<Elemento>::iterator it2;
for (it2 = v.begin(); it2 != v.end(); ++it2) {
if (it != it2) {
// No compararlo consigo mismo
float distancia = temp.distancia(*it2);
distancias_temp.push_back(make_pair(distancia, *it2));
}
}
for (int i = 0; i < vecinos && i < distancias_temp.size(); ++i) {
grafo->insertarArista(temp, distancias_temp[i].second, distancias_temp[i].first);
"El mapa se ha generado exitosamente. Cada elemento tiene " << vecinos << " vecinos" << endl;
End

ruta_mas_larga

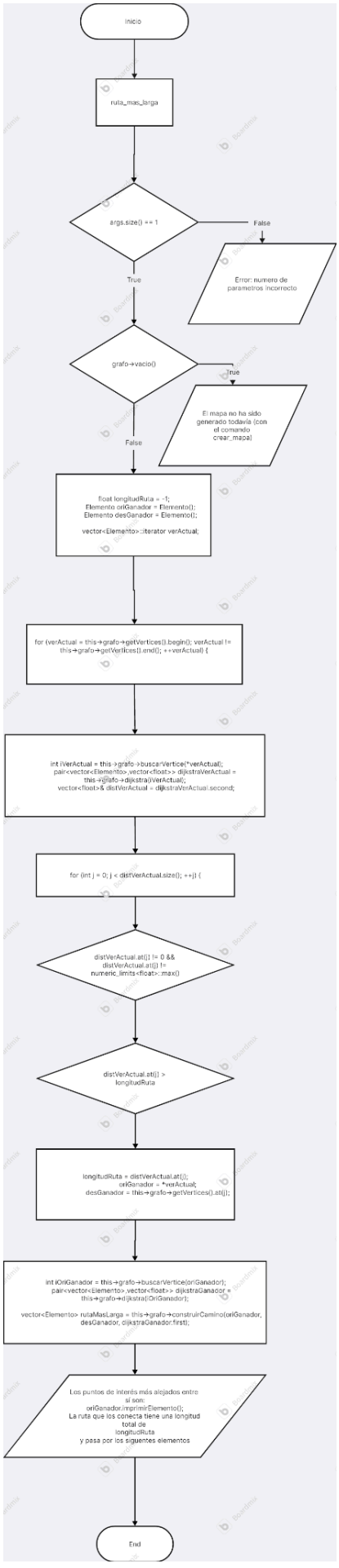
-Entradas: ruta_m as_larga
-Salidas: imprime en pantalla los elementos de origen y destino, la longitud total de la ruta, y la secuencia de elementos que hay que seguir para ir del elemento origen al elemento destino.
Aparte de esta acción muestra 7 diferentes tipos de mensajes en pantalla dependiendo de las entradas

- Error: numero de parametros incorrecto
- El mapa no ha sido generado todavía (con el comando crear_mapa)
- Los puntos de interés más alejados entre sí son:
- La ruta que los conecta tiene una longitud total de
- y pasa por los siguentes elementos
-

-Condiciones para el comando:

Se debe haber agregado elementos y creado un mapa con anterioridad.
No debe llenar más información aparte de los parámetros solicitados.
Debe seguir la estructura ruta_mas_larga

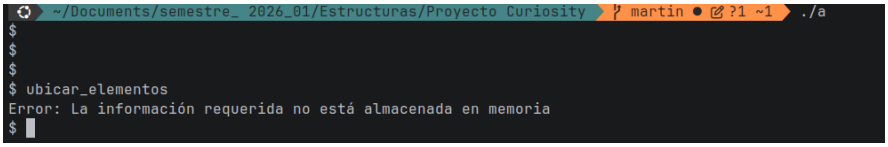
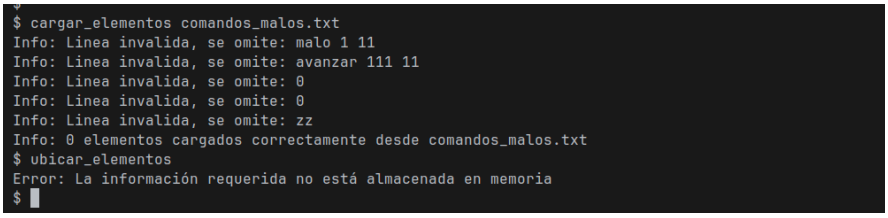
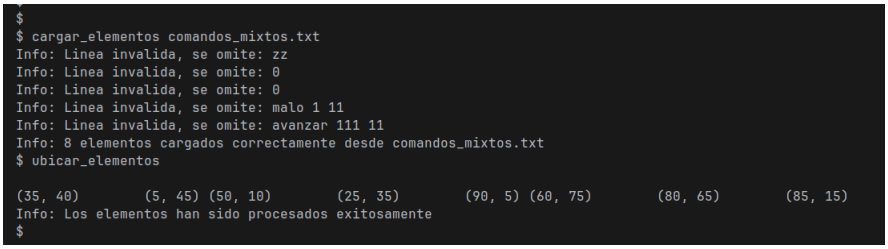
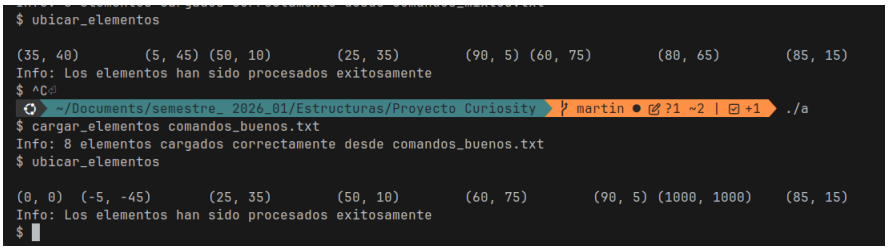
Diagrama de flujo UML


Inicio
ruta_mas_larga
args.size() == 1
False
Error: numero de parametros incorrecto
True
grafo->vacio()
True
El mapa no ha sido generado todavía (con el comando crear_mapa)
False
float longitudRuta = -1;
Elemento oriGanador = Elemento();
Elemento desGanador = Elemento();
vector<Elemento>::iterator verActual;
for (verActual = this->grafo->getVertices().begin(); verActual != this->grafo->getVertices().end(); ++verActual) {
int iVerActual = this->grafo->buscarVertice(*verActual);
pair<vector<Elemento>,vector<float>> dijkstraVerActual = this->grafo->dijkstra(iVerActual);
vector<float>& distVerActual = dijkstraVerActual.second;
for (int j = 0; j < distVerActual.size(); ++j) {
distVerActual.at(j) != 0 && distVerActual.at(j) != numeric_limits<float>::max()
distVerActual.at(j) > longitudRuta
longitudRuta = distVerActual.at(j);
oriGanador = *verActual;
desGanador = this->grafo->getVertices().at(j);
int iOriGanador = this->grafo->buscarVertice(oriGanador);
pair<vector<Elemento>,vector<float>> dijkstraGanador = this->grafo->dijkstra(iOriGanador);
vector<Elemento> rutaMasLarga = this->grafo->construirCamino(oriGanador, desGanador, dijkstraGanador.first);
Los puntos de interés más alejados entre sí son:
oriGanador.imprimirElemento();
La ruta que los conecta tiene una longitud total de
longitudRuta
y pasa por los siguientes elementos
End

Plan de pruebas comando ubicar_elementos

Descripción de caso	**Valores de entrada**	**Resultado esperado**	**Resultado obtenido**
No hay elementos almacenados en la memoria.	N/A	Error: La información requerida no está en memoria	Error: La información requerida no esta en memoria
Se cargan los elementos pero con valores inválidos	Valores en cargar elementos: malo 1 11 avanzar 111 11 0 0 zz	Error: La información requerida no está en memoria	Error: La información requerida no está en memoria
Se cargan elementos con valores inválidos y valores válidos	Valores en cargar elementos: roca 13.2 cm 35 40 zz duna 0.5 km 5 45 crater 0.55 m 25 35 monticulo 120 dm 50 10 roca 5.0 cm 90 5 duna 1.1 km 60 75 0 0 crater 250 cm 80 65 monticulo 2.2 m 85 15 malo 1 11 avanzar 111 11	$ ubicar_elementos (35, 40) (5, 45) (50, 10) (25, 35) (90, 5) (60, 75) (80, 65) (85, 15) Info: Los elementos han sido procesados exitosamente	$ ubicar_elementos (35, 40) (5, 45) (50, 10) (25, 35) (90, 5) (60, 75) (80, 65) (85, 15) Info: Los elementos han sido procesados exitosamente
Se cargan elementos positivos, negativos, en 0, pero validos	Valores en cargar elementos: roca 13.2 cm 0 0 duna 0.5 km -5 -45 crater 0.55 m 25 35 monticulo 120 dm 50 10 roca 5.0 cm 90 5 duna 1.1 km 60 75 crater 250 cm 1000	$ ubicar_elementos (0, 0) (-5, -45) (25, 35) (50, 10) (60, 75) (90, 5) (1000, 1000) (85, 15) Info: Los elementos han sido procesados exitosamente	$ ubicar_elementos (0, 0) (-5, -45) (25, 35) (50, 10) (60, 75) (90, 5) (1000, 1000) (85, 15) Info: Los elementos han sido procesados exitosamente

	1000 monticulo 2.2 m 85 15		

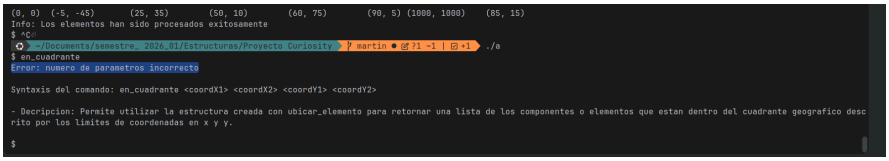
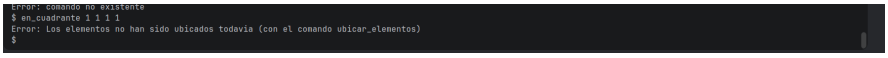
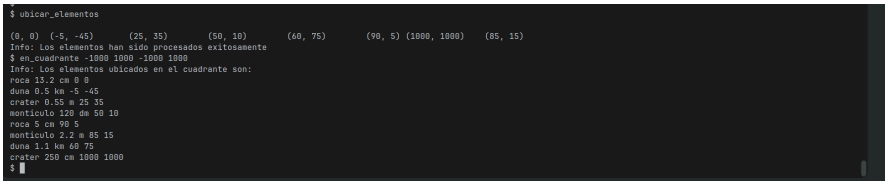
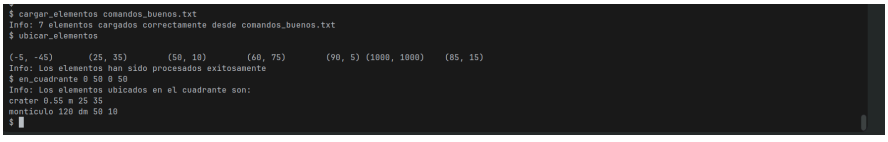
Descripción de caso	**Evidencias de ejecución**
La información requerida no está en memoria	
Se cargan los elementos pero con valores inválidos	
Se cargan elementos con valores inválidos y valores válidos	
Se cargan elementos positivos, negativos, en 0, pero válidos	

Plan de pruebas comando en_cuadrante.

Descripción de caso	**Valores de entrada**	**Resultado esperado**	**Resultado obtenido**
Numero de parametros	en_cuadrante	Error: numero de parametros	Error: numero de parametros

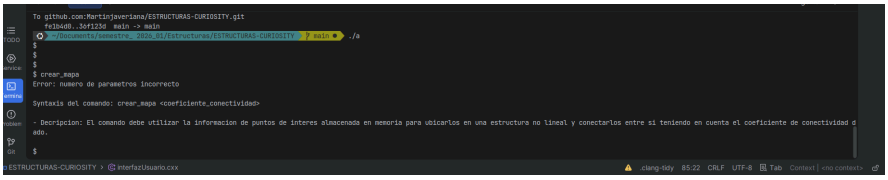
incorrecto		incorrecto	incorrecto
Numero de parametros incorrecto	N/A	Error: numero de parametros incorrecto	Error: numero de parametros incorrecto
No hay elementos ubicados	Cualquier cuadrante	Error: Los elementos no han sido ubicados todavia (con el comando ubicar_elementos)	Error: Los elementos no han sido ubicados todavia (con el comando ubicar_elementos)
Todos los valores están dentro del cuadrante	Valores en cargar elementos: roca 13.2 cm 0 0 duna 0.5 km -5 -45 crater 0.55 m 25 35 monticulo 120 dm 50 10 roca 5.0 cm 90 5 duna 1.1 km 60 75 crater 250 cm 1000 1000 monticulo 2.2 m 85 15 Valores comando En_cuadrante -1000 1000 -1000 1000	Se imprimen todos los elementos, ya que están todos en el cuadrante descrito Info: Los elementos ubicados en el cuadrante son: roca 13.2 cm 0 0 duna 0.5 km -5 -45 crater 0.55 m 25 35 monticulo 120 dm 50 10 roca 5 cm 90 5 monticulo 2.2 m 85 15 duna 1.1 km 60 75 crater 250 cm 1000 1000	Info: Los elementos ubicados en el cuadrante son: roca 13.2 cm 0 0 duna 0.5 km -5 -45 crater 0.55 m 25 35 monticulo 120 dm 50 10 roca 5 cm 90 5 monticulo 2.2 m 85 15 duna 1.1 km 60 75 crater 250 cm 1000 1000
No hay ningún elemento dentro del cuadrante	Valores en cargar elementos: duna 0.5 km -5 -45 crater 0.55 m 25 35 monticulo 120 dm 50 10 roca 5.0 cm 90 5 duna 1.1 km 60 75 crater 250 cm 1000 1000 monticulo 2.2 m 85 15 Valores comando En_cuadrante 0 0 0 0	no hay elementos en el cuadrante	no hay elementos en el cuadrante
Hay algunos	Valores en cargar	Info: Los elementos	Info: Los elementos

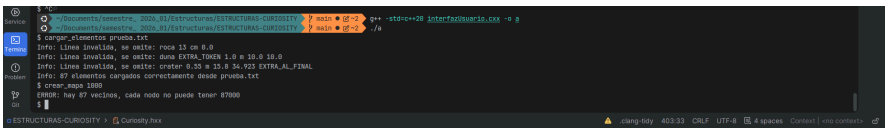
elementos en el cuadrante	elementos: duna 0.5 km -5 -45 crater 0.55 m 25 35 monticulo 120 dm 50 10 roca 5.0 cm 90 5 duna 1.1 km 60 75 crater 250 cm 1000 1000 monticulo 2.2 m 85 15 Valores comando En_cuadrante 0 50 0 50	ubicados en el cuadrante son: crater 0.55 m 25 35 monticulo 120 dm 50 10	ubicados en el cuadrante son: crater 0.55 m 25 35 monticulo 120 dm 50 10

Descripción de caso	**Evidencias de ejecución**
Numero de parametros incorrecto	
No hay elementos ubicados	
Todos los valores están dentro del cuadrante	
No hay valores dentro del cuadrante	
Hay algunos valores dentro del cuadrante	

Plan de pruebas comando crear_mapa .

Descripción de caso	Valores de entrada	Resultado esperado	Resultado obtenido
Numero de parametros incorrecto	crear_mapa	Error: numero de parametros incorrecto	Error: numero de parametros incorrecto
Tipo de parámetro incorrecto	crear_mapa [cualquier carácter]	Error: el coeficiente tiene que ser un número	Error: el coeficiente tiene que ser un número
No hay elementos en memoria y los parámetros son correctos	crear_mapa 0.5	Error: (No hay información) La información requerida no está almacenada en memoria.	Error:(No hay información) La información requerida no está almacenada en memoria.
Se pide que cada nodo tenga más vecinos de los que puede tener	crear_mapa 1000	Error: hay n vecinos, cada nodo no puede tener n*1000 vecinos	Error: hay n vecinos, cada nodo no puede tener n*1000 vecinos
Caso de exito	crear_mapa 0.5	El mapa se ha generado exitosamente. Cada elemento tiene n*0.5	El mapa se ha generado exitosamente. Cada elemento tiene n*0.5

Descripción de caso	Evidencias de ejecución
Numero de parametros incorrecto	
Tipo de parámetro incorrecto	
No hay elementos en memoria y los parámetros son correctos	

Se pide que cada nodo tenga más vecinos de los que puede tener	
Caso de exito	

Plan de pruebas comando ruta_mas_larga .

Descripción de caso	**Valores de entrada**	**Resultado esperado**	**Resultado obtenido**
Numero de parametros incorrecto	ruta_mas_larga [Cualquier numero]	Error: numero de parametros incorrecto	Error: numero de parametros incorrecto
Numero de parametros incorrecto	ruta_mas_larga [Cualquier palabra]	Error: numero de parametros incorrecto	Error: numero de parametros incorrecto
El mapa no ha sido generado anterior al comando	ruta_mas_larga	Error: El mapa no ha sido generado todavía (con el comando crear_mapa).	Error: El mapa no ha sido generado todavía (con el comando crear_mapa).
Caso de exito	Ruta_mas_larga Elementos ubicados en el mapa: roca 13 cm 0.0 0.0 duna 0.5 m 1.5 2.3 crater 0.55 m 3.1 4.8 monticulo 2.1 m 5.0 7.2 roca 45 mm 7.3 9.1	Los puntos de interés más alejados entre sí son: tipo: duna tamaño: 0.5 unidad: km coordenada x:24.345 coordenada y: 67.456 y tipo: roca tamaño: 13 unidad: cm coordenada x:3.45 coordenada y: 15.4 La ruta que los conecta tiene una longitud total de 56.7378	Los puntos de interés más alejados entre sí son: tipo: duna tamaño: 0.5 unidad: km coordenada x:24.345 coordenada y: 67.456 y tipo: roca tamaño: 13 unidad: cm coordenada x:3.45 coordenada y: 15.4 La ruta que los conecta tiene una longitud total de 56.7378

No hay ningún elemento en memoria	Crear_mapa Caso anterior al commando para asegurar que hay elementos en el mapa	(No hay información) La información requerida no está almacenada en memoria.	(No hay información) La información requerida no está almacenada en memoria.

Descripción de caso	**Evidencias de ejecución**
Numero de parametros incorrecto	
El mapa no ha sido generado anterior al comando	
Caso de exito	
No hay ningún elemento en memoria	